

C++ Standard Library Issues to be moved in Virtual Plenary, Feb. 2022

Doc. no. P2531R0

Date: 2022-01-31

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

Tentatively Ready Issues

[3088](#). `forward_list::merge` behavior unclear when passed `*this`

Section: 22.3.9.6 [[forward.list.ops](#)] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2018-03-19 **Last modified:** 2022-01-31

Priority: 3

Discussion:

LWG [300](#) changed `list::merge` to be a no-op when passed `*this`, but there's no equivalent rule for `forward_list::merge`. Presumably the `forward_list` proposal predated the adoption of LWG 300's PR and was never updated for the change. Everything in the discussion of that issue applies mutatis mutandis to the current specification of `forward_list::merge`.

[2018-06-18 after reflector discussion]

Priority set to 3

[2019-07-30 Tim provides updated PR]

Per the comments during issue prioritization, the new PR tries to synchronize the wording between `list::merge` and `forward_list::merge`.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4727](#).

1. Edit 99 [[forwardlist.ops](#)] as indicated:

```
void merge(forward_list& x);
void merge(forward_list&& x);
template<class Compare> void merge(forward_list& x, Compare comp);
template<class Compare> void merge(forward_list&& x, Compare comp);
```

-20- *Requires:* `*this` and `x` are both sorted with respect to the comparator `operator<` (for the first two overloads) or `comp` (for the last two overloads), and `get_allocator() == x.get_allocator()` is true.

-21- *Effects:* If `addressof(x) == this`, does nothing. Otherwise, merges the two sorted ranges `[begin(), end())` and `[x.begin(), x.end())`. The result is a range that is sorted with respect to the comparator `operator<` (for the first two overloads) or `comp` (for the last two overloads). `x` is empty after the merge. If an exception is thrown other than by a comparison there are no effects. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

-22- *Remarks:* Stable (16.4.6.8 [[algorithm.stable](#)]). The behavior is undefined if `get_allocator() != x.get_allocator()`.

-23- *Complexity:* At most `distance(begin(), end()) + distance(x.begin(), x.end()) - 1` comparisons if `addressof(x) != this`; otherwise, no comparisons are performed.

[2021-05-22 Tim syncs wording to the current working draft]

[2022-01-31; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4901](#).

1. Edit 22.3.9.6 [\[forward.list.ops\]](#) as indicated:

```
void merge(forward_list& x);
void merge(forward_list&& x);
template<class Compare> void merge(forward_list& x, Compare comp);
template<class Compare> void merge(forward_list&& x, Compare comp);
```

-?- Let comp be less<>{} for the first two overloads.

-24- Preconditions: ~~*this and x are both sorted with respect to the comparator~~ **operator< (for the first two overloads) or comp (for the last two overloads),** and ~~get_allocator() == x.get_allocator() is true.~~

-25- Effects: **If addressof(x) == this, there are no effects. Otherwise, merges the two sorted ranges [begin(), end()) and [x.begin(), x.end()). The result is a range that is sorted with respect to the comparator comp. x is empty after the merge. If an exception is thrown other than by a comparison there are no effects.** Pointers and references to the moved elements of x now refer to those same elements but as members of *this. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into *this, not into x.

-26- Complexity: At most distance(begin(), end()) + distance(x.begin(), x.end()) - 1 comparisons **if addressof(x) != this; otherwise, no comparisons are performed.**

-27- Remarks: Stable (16.4.6.8 [\[algorithm.stable\]](#)). **If addressof(x) != this, x is empty after the merge. No elements are copied by this operation. If an exception is thrown other than by a comparison there are no effects.**

2. Edit 22.3.10.5 [\[list.ops\]](#) as indicated:

```
void merge(list& x);
void merge(list&& x);
template<class Compare> void merge(list& x, Compare comp);
template<class Compare> void merge(list&& x, Compare comp);
```

-?- Let comp be less<>{} for the first two overloads.

-26- Preconditions: ~~Both the list and the argument list shall be~~ ***this and x are both** sorted with respect to the comparator ~~operator< (for the first two overloads) or comp (for the last two overloads),~~ and ~~get_allocator() == x.get_allocator() is true.~~

-27- Effects: ~~If addressof(x) == this, does nothing; otherwise, merges the two sorted ranges [begin(), end()) and [x.begin(), x.end()). The result is a range in which the elements will be sorted in non-decreasing order according to the ordering defined by comp; that is, for every iterator i, in the range other than the first, the condition comp(*i, *(i - 1)) will be false.~~ **that is sorted with respect to the comparator comp.** Pointers and references to the moved elements of x now refer to those same elements but as members of *this. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into *this, not into x.

-28- Complexity: At most size() + x.size() - 1 ~~applications of comp~~ **comparisons** if addressof(x) != this; otherwise, ~~no applications of comp~~ **comparisons** are performed. ~~If an exception is thrown other than by a comparison there are no effects.~~

-29- Remarks: Stable (16.4.6.8 [\[algorithm.stable\]](#)). ~~If addressof(x) != this, the range [x.begin(), x.end()) is empty after the merge. No elements are copied by this operation.~~ **If an exception is thrown other than by a comparison there are no effects.**

[3471](#). `polymorphic_allocator::allocate` does not satisfy *Cpp17Allocator* requirements

Section: 20.12 [\[mem.res\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Alisdair Meredith **Opened:** 2020-07-27 **Last modified:** 2022-01-31

Priority: 3

View all other [issues](#) in [\[mem.res\]](#).

Discussion:

With the adoption of [P0593R6](#) in Prague, `std::ptr::polymorphic_allocator` no longer satisfies the of *Cpp17Allocator* requirements. Specifically, all calls to `allocate(n)` need to create an object for an array of `n` Ts (but not initialize any of those elements). `std::pmr::polymorphic_allocator` calls its underlying memory resource to allocate sufficient bytes of storage, but it does not create (and start the lifetime of) the array object within that storage.

[2020-08-03; Billy comments]

It's worth noting that the resolution of [CWG 2382](#) has impact on implementors for this issue.

[2020-08-21; Reflector prioritization]

Set priority to 3 after reflector discussions.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4861](#).

[Drafting note: The proposed wording is inspired by the example given in the "Assertion/note" column of the expression `a.allocate(n)` in table [tab:cpp17 allocator].]

1. Modify 20.12.2.2 [mem.res.public] as indicated:

```
[[nodiscard]] void* allocate(size_t bytes, size_t alignment = max_align);
```

-2- Effects: Equivalent to: ~~return do_allocate(bytes, alignment);~~

```
void* p = do_allocate(bytes, alignment);  
return launder(new (p) byte[bytes]);
```

[2021-05-20 Tim comments and updates wording]

`memory_resource::allocate` is the PMR equivalent of `malloc` and operator `new`. It therefore needs the "suitable created object" wording (see 6.7.2 [intro.object], 20.10.11 [c.malloc]). This ensures that it creates the requisite array object automatically for all the allocation functions of `polymorphic_allocator`, and returns the correct pointer value in all cases.

[2022-01-31; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4885](#).

1. Modify 20.12.2.2 [mem.res.public] as indicated:

```
[[nodiscard]] void* allocate(size_t bytes, size_t alignment = max_align);
```

-2- Effects: ~~Equivalent to: return~~ Allocates storage by calling `do_allocate(bytes, alignment)`; ~~and implicitly creates objects within the allocated region of storage.~~

~~.- Returns: A pointer to a suitable created object (6.7.2 [intro.object]) in the allocated region of storage.~~

~~.- Throws: What and when the call to `do_allocate` throws.~~

[3525](#). `uses_allocator_construction_args` fails to handle types convertible to pair

Section: 20.10.7.2 [allocator.uses.construction] Status: [Tentatively Ready](#) Submitter: Tim Song Opened: 2021-02-23 Last modified: 2022-01-31

Priority: 3

View other [active issues](#) in [allocator.uses.construction].

View all other [issues](#) in [allocator.uses.construction].

Discussion:

As currently specified, the following program is ill-formed (and appears to have been since LWG [2975](#)):

```
struct S {  
    operator std::pair<const int, int>() const {  
        return {};  
    }  
};  
  
void f() {  
    std::pmr::map<int, int> s;  
    s.emplace(S{});  
}
```

There's no matching overload for `uses_allocator_construction_args<pair<const int, int>>(alloc, S&&)`, since `S` is not a pair and every overload for constructing pairs that takes one non-allocator argument expects a pair from which template arguments can be deduced.

[2021-02-27 Tim adds PR and comments]

The superseded resolution below attempts to solve this issue by adding two additional overloads of `uses_allocator_construction_args` to handle this case. However, the new overloads forces implicit conversions at the call to `uses_allocator_construction_args`, which requires the result to be

consumed within the same full-expression before any temporary created from the conversion is destroyed. This is not the case for the `piecewise_construct` overload of `uses_allocator_construction_args`, which recursively calls `uses_allocator_construction_args` for the two elements of the pair, which might themselves be pairs.

The approach taken in the revised PR is to produce an exposition-only *pair-constructor* object instead. The object holds the allocator and the argument by reference, implicitly converts to the specified specialization of `pair`, and when so converted return a pair that is constructed by `uses_allocator` construction with the converted value of the original argument. This maintains the existing design that `pair` itself doesn't know anything about allocator construction.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4878](#).

1. Edit 20.10.2 [\[memory.syn\]](#), header `<memory>` synopsis, as indicated:

```
namespace std {
    [...]
    // 20.10.7.2 \[allocator.uses.construction\], uses-allocator construction
    [...]

    template<class T, class Alloc>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    const remove_cv_t<T>& pr) noexcept;

    template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    const pair<U, V>& pr) noexcept -> see below;

    template<class T, class Alloc>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    remove_cv_t<T>&& pr) noexcept;

    template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    pair<U, V>&& pr) noexcept -> see below;

    [...]
}
```

2. Edit 20.10.7.2 [\[allocator.uses.construction\]](#) as indicated:

```
template<class T, class Alloc>
constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                const remove_cv_t<T>& pr) noexcept;

template<class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                const pair<U, V>& pr) noexcept -> see below;

-12- Constraints: T is a specialization of pair. For the second overload, is same v<pair<U, V>,
remove_cv_t<T>> is false.

-13- Effects: Equivalent to:

    return uses_allocator_construction_args<T>(alloc, piecewise_construct,
                                                forward_as_tuple(pr.first),
                                                forward_as_tuple(pr.second));

template<class T, class Alloc>
constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                remove_cv_t<T>&& pr) noexcept;

template<class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                pair<U, V>&& pr) noexcept -> see below;

-14- Constraints: T is a specialization of pair. For the second overload, is same v<pair<U, V>,
remove_cv_t<T>> is false.

-15- Effects: Equivalent to:

    return uses_allocator_construction_args<T>(alloc, piecewise_construct,
                                                forward_as_tuple(std::move(pr).first),
                                                forward_as_tuple(std::move(pr).second));
```

[2021-03-12; Reflector poll]

Set priority to 3 following reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4878](#).

1. Edit 20.10.2 [\[memory.syn\]](#), header <memory> synopsis, as indicated:

```
namespace std {
    [...]
    // 20.10.7.2 \[allocator.uses.construction\], uses-allocator construction
    [...]

    template<class T, class Alloc, class U, class V>
        constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                         const pair<U, V>& pr) noexcept -> see below;

    template<class T, class Alloc, class U, class V>
        constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                         pair<U, V>&& pr) noexcept -> see below;

    template<class T, class Alloc, class U>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc, U&& u) noexcept;
    [...]
}
```

2. Add the following to 20.10.7.2 [\[allocator.uses.construction\]](#):

```
template<class T, class Alloc, class U>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, U&& u) noexcept;

-?- Let FUN be the function template:

    template<class A, class B>
    void FUN(const pair<A, B>&);

-?- Constraints: T is a specialization of pair, and the expression FUN(u) is not well-formed when
considered as an unevaluated operand.

-?- Effects: Equivalent to:

    return make_tuple(pair-constructor{alloc, u});

where pair-constructor is an exposition-only class defined as follows:

    struct pair-constructor {
    using pair-type = remove_cv_t<T>; // exposition only
    constexpr operator pair-type() const {
    return do_construct(std::forward<U>(u));
    };

    constexpr auto do_construct(const pair-type& p) const { // exposition only
    return make_obj_using_allocator<pair-type>(alloc, p);
    };
    constexpr auto do_construct(pair-type&& p) const { // exposition only
    return make_obj_using_allocator<pair-type>(alloc, std::move(p));
    };

    const Alloc& alloc; // exposition only
    U&& u; // exposition only
    };
```

[2021-12-02 Tim updates PR to avoid public exposition-only members]

[2022-01-31; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4901](#).

1. Edit 20.10.2 [\[memory.syn\]](#), header <memory> synopsis, as indicated:

```
namespace std {
    [...]
    // 20.10.7.2 \[allocator.uses.construction\], uses-allocator construction
    [...]
    template<class T, class Alloc, class U, class V>
        constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                         pair<U, V>& pr) noexcept;

    template<class T, class Alloc, class U, class V>
```

```

constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                const pair<U, V>& pr) noexcept;

template<class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                pair<U, V>&& pr) noexcept;

template<class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                const pair<U, V>&& pr) noexcept;

template<class T, class Alloc, class U>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, U&& u) noexcept;
[...]
```

2. Add the following to 20.10.7.2 [\[allocator.uses.construction\]](#):

```

template<class T, class Alloc, class U>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, U&& u) noexcept;
```

-?- Let *FUN* be the function template:

```

template<class A, class B>
void FUN(const pair<A, B>&);
```

-?- *Constraints*: *T* is a specialization of *pair*, and the expression *FUN(u)* is not well-formed when considered as an unevaluated operand.

-?- Let *pair-constructor* be an exposition-only class defined as follows:

```

class pair-constructor {
    using pair-type = remove_cv_t<T>; // exposition only

    constexpr auto do-construct(const pair-type& p) const { // exposition only
        return make_obj_using_allocator<pair-type>(alloc, p);
    };
    constexpr auto do-construct(pair-type&& p) const { // exposition only
        return make_obj_using_allocator<pair-type>(alloc, std::move(p));
    };

    const Alloc& alloc; // exposition only
    U& u; // exposition only

public:
    constexpr operator pair-type() const {
        return do-construct(std::forward<U>(u));
    };
};
```

-?- *Returns*: make_tuple(*pc*), where *pc* is a *pair-constructor* object whose *alloc* member is initialized with *alloc* and whose *u* member is initialized with *u*.

3598. system_category().default_error_condition(0) is underspecified

Section: 19.5.3.5 [\[syserr.errcat.objects\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2021-09-23 **Last modified:** 2021-10-14

Priority: Not Prioritized

View all other [issues](#) in [\[syserr.errcat.objects\]](#).

Discussion:

19.5.3.5 [\[syserr.errcat.objects\]](#) says:

If the argument *ev* corresponds to a POSIX *errno* value *posv*, the function shall return `error_condition(posv, generic_category())`. Otherwise, the function shall return `error_condition(ev, system_category())`. What constitutes correspondence for any given operating system is unspecified.

What is "a POSIX *errno* value"? Does it mean a value equal to one of the `<cerrno> Exxx` macros? Because in that case, the value 0 is not "a POSIX *errno* value". So arguably `system_category().default_error_condition(0)` is required to return an `error_condition` using the "system" category, which means that `error_code{} == error_condition{}` is required to be false. This seems wrong.

For POSIX-based systems the value 0 should definitely correspond to the generic category. Arguably that needs to be true for all systems, because the `std::error_code` API strongly encourages a model where zero means "no error".

The proposed resolution has been implemented in libstdc++. Libc++ has always treated system error code 0 as corresponding to generic error code 0.

[2021-10-14; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4892](#).

1. Modify 19.5.3.5 [\[syserr.errcat.objects\]](#) as indicated:

```
const error_category& system_category() noexcept;
```

-3- *Returns*: [...]

-4- *Remarks*: The object's equivalent virtual functions shall behave as specified for class `error_category`. The object's name virtual function shall return a pointer to the string "system". The object's `default_error_condition` virtual function shall behave as follows:

If the argument `ev` is equal to 0, the function returns `error_condition(0, generic_category())`. Otherwise, if `ev` corresponds to a POSIX `errno` value `posv`, the function shall return `error_condition(posv, generic_category())`. Otherwise, the function shall return `error_condition(ev, system_category())`. What constitutes correspondence for any given operating system is unspecified.

[Note 1: The number of potential system error codes is large and unbounded, and some might not correspond to any POSIX `errno` value. Thus implementations are given latitude in determining correspondence. — end note]

3601. `common_iterator`'s *postfix-proxy* needs indirectly_readable

Section: 23.5.4.5 [\[common.iter.nav\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2021-09-24 **Last modified:** 2021-10-14

Priority: Not Prioritized

View all other issues in [\[common.iter.nav\]](#).

Discussion:

It would appear that when [P2259R1](#) added *postfix-proxy* to `common_iterator::operator++(int)` LWG missed a crucial difference between `operator++(int)` and `operator->` which uses a similar proxy: `operator->` requires the wrapped type to be `indirectly_readable`, but `operator++(int)` does not. Consequently, operations that read from the wrapped type for the *postfix-proxy* case in `operator++(int)` are not properly constrained to be valid.

[2021-10-14; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4892](#).

1. Modify 23.5.4.5 [\[common.iter.nav\]](#) as indicated:

```
decltype(auto) operator++(int);
```

-4- *Preconditions*: `holds_alternative<I>(v_)` is true.

-5- *Effects*: If `I` models `forward_iterator`, equivalent to:

```
common_iterator tmp = *this;
++*this;
return tmp;
```

Otherwise, if `requires (I& i) { { *i++ } -> can-reference; } is true` or `indirectly_readable<I> && constructible_from<iter_value_t<I>, iter_reference_t<I>> && move_constructible<iter_value_t<I>> is false`, equivalent to:

```
return get<I>(v_++);
```

Otherwise, equivalent to: [...]

3607. `contiguous_iterator` should not be allowed to have custom `iter_move` and `iter_swap` behavior

Section: 23.3.4.14 [\[iterator.concept.contiguous\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2021-09-28 **Last modified:** 2021-10-23

Priority: Not Prioritized

Discussion:

The `iter_move` and `iter_swap` customization points were introduced primarily for proxy iterators. Whatever their application to non-proxy iterators in general, they should not be allowed to have custom behavior for contiguous iterators — this new iterator category was introduced in large part to permit better optimizations, and allowing custom `iter_move/iter_swap` prevents such optimizations for a wide variety of algorithms that are specified to call them.

[2021-10-14; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4892](#).

1. Modify 23.3.4.14 [\[iterator.concept.contiguous\]](#) as indicated:

```
template<class I>
concept contiguous_iterator =
    random_access_iterator<I> &&
    derived_from<ITER_CONCEPT(I), contiguous_iterator_tag> &&
    is_lvalue_reference_v<iter_reference_t<I>> &&
    same_as<iter_value_t<I>, remove_cvref_t<iter_reference_t<I>>> &&
    requires(const I& i) {
        { to_address(i) } -> same_as<add_pointer_t<iter_reference_t<I>>>;
    };

```

-2- Let `a` and `b` be dereferenceable iterators and `c` be a non-dereferenceable iterator of type `I` such that `b` is reachable from `a` and `c` is reachable from `b`, and let `D` be `iter_difference_t<I>`. The type `I` models `contiguous_iterator` only if

(2.1) — `to_address(a) == addressof(*a)`,

(2.2) — `to_address(b) == to_address(a) + D(b - a)`, **and**

(2.3) — `to_address(c) == to_address(a) + D(c - a)`.

(2.2) — `ranges::iter_move(a)` has the same type, value category, and effects as `std::move(*a)`, and

(2.2) — if `ranges::iter_swap(a, b)` is well-formed, it has effects equivalent to `ranges::swap(*a, *b)`.

3610. `iota_view::size` sometimes rejects integer-class types

Section: 24.6.4.2 [\[range.iota.view\]](#) Status: [Tentatively Ready](#) Submitter: Jiang An Opened: 2021-09-29 Last modified: 2021-10-14

Priority: Not Prioritized

View other [active issues](#) in [\[range.iota.view\]](#).

View all other [issues](#) in [\[range.iota.view\]](#).

Discussion:

It seems that the `iota_view` tends to accept integer-class types as its value types, by using *is-integer-like* or *is-signed-integer-like* through the specification, although it's unspecified whether any of them satisfies `weakly_incrementable`. However, the *requires-clause* of `iota_view::size` (24.6.4.2 [\[range.iota.view\]](#) p16) uses `(integral<W> && integral<Bound>)`, which sometimes rejects integer-class types.

Should we relax the restrictions by changing this part to `(is-integer-like<W> && is-integer-like<Bound>)`?

[2021-10-14; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4892](#).

1. Modify 24.6.4.2 [\[range.iota.view\]](#) as indicated:

```
constexpr auto size() const requires see below;

```

-15- Effects: Equivalent to:

```
if constexpr (is-integer-like<W> && is-integer-like<Bound>)
    return (value_ < 0)
        ? ((bound_ < 0)
            ? to-unsigned-like(-value_) - to-unsigned-like(-bound_)
            : to-unsigned-like(bound_) + to-unsigned-like(-value_))

```



```

        : to-unsigned-like(bound_) - to-unsigned-like(value_);
    else
        return to-unsigned-like(bound_ - value_);

```

-16- *Remarks:* The expression in the *requires-clause* is equivalent to:

```

(same_as<W, Bound> && advanceable<W>) || (integralis-integer-like<W> && integralis-integer-like<Bound>) ||
    sized_sentinel_for<Bound, W>

```

3612. Inconsistent pointer alignment in `std::format`

Section: 20.20.2.2 [\[format.string.std\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Victor Zverovich **Opened:** 2021-10-02 **Last modified:** 2021-10-14

Priority: Not Prioritized

View other [active issues](#) in `[format.string.std]`.

View all other [issues](#) in `[format.string.std]`.

Discussion:

According to `[tab:format.type.ptr]` pointers are formatted as hexadecimal integers (at least in the common case when `uintptr_t` is available). However, it appears that they have left alignment by default according to `[tab:format.align]`:

Forces the field to be aligned to the start of the available space. This is the default for non-arithmetic types, `charT`, and `bool`, unless an integer presentation type is specified.

because pointers are not arithmetic types.

For example:

```

void* p = ...
std::format("{:#16x}", std::bit_cast<uintptr_t>(p));
std::format("{:16}", p);

```

may produce " 0x7fff88716c84" and "0x7fff88716c84 " (the actual output depends on the value of `p`).

This is inconsistent and clearly a bug in specification that should have included pointers together with arithmetic types in `[tab:format.align]`.

[2021-10-14; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4892](#).

1. Modify 20.20.2.2 [\[format.string.std\]](#), Table `[tab:format.align]`, as indicated:

Table 59 — Meaning of *align* options `[tab:format.align]`

Option	Meaning
<	Forces the field to be aligned to the start of the available space. This is the default for non-arithmetic non-pointer types, <code>charT</code> , and <code>bool</code> , unless an integer presentation type is specified.
>	Forces the field to be aligned to the end of the available space. This is the default for arithmetic types other than <code>charT</code> and <code>bool</code> , pointer types or when an integer presentation type is specified.
[...]	

[Drafting note: The wording above touches a similar area as LWG [3586](#). To help solving the merge conflict the following shows the delta of this proposed wording on top of the LWG [3586](#) merge result]

Table 59 — Meaning of *align* options `[tab:format.align]`

Option	Meaning
<	Forces the field to be aligned to the start of the available space. This is the default when the presentation type is a non-arithmetic non-pointer type.
>	Forces the field to be aligned to the end of the available space. This is the default when the presentation type is an arithmetic or pointer type.
[...]	

3616. LWG 3498 seems to miss the non-member swap for `basic_syncbuf`

Section: 29.11.2.6 [\[syncstream.syncbuf.special\]](#) **Status:** [Tentatively Ready](#) **Submitter:** S. B. Tam **Opened:** 2021-10-07 **Last modified:** 2021-10-14

Priority: Not Prioritized

Discussion:

LWG [3498](#) fixes the inconsistent noexcept-specifiers for member functions of `basic_syncbuf`, but the proposed resolution in LWG [3498](#) seems to miss the non-member `swap`, which also has inconsistent noexcept-specifier: 29.11.2.6 [\[syncstream.syncbuf.special\]](#) says it's noexcept, while 29.11.1 [\[syncstream.syn\]](#) says it's not.

Since the non-member `swap` and the member `swap` have equivalent effect, and LWG 3498 removes noexcept from the latter, I think it's pretty clear that the former should not be noexcept.

[2021-10-14; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4892](#).

1. Modify 29.11.2.6 [\[syncstream.syncbuf.special\]](#) as indicated:

```
template<class charT, class traits, class Allocator>
void swap(basic_syncbuf<charT, traits, Allocator>& a,
         basic_syncbuf<charT, traits, Allocator>& b) noexcept;
```

-1- *Effects:* Equivalent to `a.swap(b)`.

[3618](#). Unnecessary `iter_move` for `transform_view::iterator`

Section: 24.7.7.3 [\[range.transform.iterator\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Barry Revzin **Opened:** 2021-10-12 **Last modified:** 2021-10-17

Priority: Not Prioritized

View other [active issues](#) in `[range.transform.iterator]`.

View all other [issues](#) in `[range.transform.iterator]`.

Discussion:

`transform_view`'s iterator currently specifies a customization point for `iter_move`. This customization point does the same thing that the default implementation would do — `std::move(*E)` if `*E` is an lvalue, else `*E` — except that it is only there to provide the correct conditional noexcept specification. But for `iota_view`, we instead provide a noexcept-specifier on the iterator's operator*. We should do the same thing for both, and the simplest thing would be:

Change 24.7.7.3 [\[range.transform.iterator\]](#) to put the whole body into noexcept:

```
constexpr decltype(auto) operator*() const noexcept(noexcept(involve(*parent_ ->fun_, *current_))) {
    return involve(*parent_ ->fun_, *current_);
}
```

And to remove this:

```
friend constexpr decltype(auto) iter_move(const iterator& i)
noexcept(noexcept(involve(*i.parent_ ->fun_, *i.current_))) {
    if constexpr (is_lvalue_reference_v<decltype(*i)>)
        return std::move(*i);
    else
        return *i;
}
```

[2022-01-29; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4892](#).

1. Modify 24.7.7.3 [\[range.transform.iterator\]](#), class template `transform_view::iterator`, as indicated:

```
[...]
constexpr decltype(auto) operator*() const noexcept(noexcept(involve(*parent_ ->fun_, *current_))) {
    return involve(*parent_ ->fun_, *current_);
}
[...]
```

```
friend constexpr decltype(auto) iter_move(const iterator& i)
noexcept(noexcept(invoker(*i.parent->fun, *i.current))) {
if constexpr (is_lvalue_reference_v<decltype(*i)>)
return std::move(*i);
else
return *i;
}
[...]
```

3619. Specification of vformat_to contains ill-formed formatted_size calls

Section: 20.20.5 [\[format.functions\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2021-10-17 **Last modified:** 2021-10-17

Priority: Not Prioritized

View all other [issues](#) in [\[format.functions\]](#).

Discussion:

The specification of vformat_to says that it formats "into the range [out, out + N), where N is formatted_size(fmt, args...) for the functions without a loc parameter and formatted_size(loc, fmt, args...) for the functions with a loc parameter".

This is wrong in at least two ways:

- First, args is a (w)format_args, not a pack, so it doesn't make sense to use ... to expand it.
- Second, fmt is a (w)string_view parameter, and it is never a constant expression, and so the call is ill-formed after [P2216](#) added compile-time format string checking.

[2022-01-29; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4892](#).

1. Modify 20.20.5 [\[format.functions\]](#) as indicated:

```
template<class Out>
Out vformat_to(Out out, string_view fmt, format_args args);
template<class Out>
Out vformat_to(Out out, wstring_view fmt, wformat_args args);
template<class Out>
Out vformat_to(Out out, const locale& loc, string_view fmt, format_args args);
template<class Out>
Out vformat_to(Out out, const locale& loc, wstring_view fmt, wformat_args args);
```

-12- Let charT be decltype(fmt)::value_type.

-13- *Constraints:* Out satisfies output_iterator<const charT>.

-14- *Preconditions:* Out models output_iterator<const charT>.

-15- *Effects:* Places the character representation of formatting the arguments provided by args, formatted according to the specifications given in fmt, into the range [out, out + N), where N is formatted_size(fmt, args...) for the functions without a loc parameter and formatted_size(loc, fmt, args...) for the functions with a loc parameter. **the number of characters in that character representation**. If present, loc is used for locale-specific formatting.

-16- *Returns:* out + N.

-17- [...]

3621. Remove feature-test macro __cpp_lib_monadic_optional

Section: 17.3.2 [\[version.syn\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jens Maurer **Opened:** 2021-10-18 **Last modified:** 2021-10-23

Priority: Not Prioritized

View other [active issues](#) in [\[version.syn\]](#).

View all other [issues](#) in [\[version.syn\]](#).

Discussion:

[P0798R8](#) "Monadic operations for `std::optional`" created a new feature-test macro `__cpp_lib_monadic_optional` for a relatively minor enhancement.

We should instead increment the value of the existing feature-test macro `__cpp_lib_optional`.

[2022-01-29; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 17.3.2 [\[version.syn\]](#) as indicated:

```
[...]
#define __cpp_lib_monadic_optional 202110L // also in <optional>
[...]
#define __cpp_lib_optional 202106L202110L // also in <optional>
[...]
```

[3632](#). `unique_ptr` "Mandates: This constructor is not selected by class template argument deduction"

Section: 20.11.1.3.2 [\[unique.ptr.single.ctor\]](#) Status: [Tentatively Ready](#) Submitter: Arthur O'Dwyer Opened: 2021-11-03 Last modified: 2021-11-06

Priority: Not Prioritized

View all other [issues](#) in `[unique.ptr.single.ctor]`.

Discussion:

[P1460R1](#) changed the wording for `unique_ptr`'s constructor `unique_ptr(pointer)`. In C++17, it said in `[unique.ptr.single.ctor]` p5:

```
explicit unique_ptr(pointer p) noexcept;
```

Preconditions: [...]

Effects: [...]

Postconditions: [...]

Remarks: If `is_pointer_v<deleter_type>` is true or `is_default_constructible_v<deleter_type>` is false, this constructor shall not participate in overload resolution. **If class template argument deduction would select the function template corresponding to this constructor, then the program is ill-formed.**

In C++20, it says in `[unique.ptr.single.ctor]` p5:

```
explicit unique_ptr(pointer p) noexcept;
```

Constraints: `is_pointer_v<deleter_type>` is false and `is_default_constructible_v<deleter_type>` is true.

Mandates: This constructor is not selected by class template argument deduction.

Preconditions: [...]

Effects: [...]

Postconditions: [...]

Normally, we use "Mandates:" for `static_assert`-like stuff, not just to indicate that some constructor doesn't contribute to CTAD. Both `libstdc++` and Microsoft (and soon `libc++`, see [LLVM issue](#)) seem to agree about the intent of this wording: It's basically asking for the constructor to be implemented with a CTAD firewall, as

```
explicit unique_ptr(type_identity_t<pointer> p) noexcept;
```

and there is no actual `static_assert` corresponding to this "Mandates:" element. In particular, the following program is well-formed on all vendors:

```
// godbolt link
template<class T> auto f(T p) -> decltype(std::unique_ptr(p));
template<class T> constexpr bool f(T p) { return true; }
static_assert(f((int*)nullptr));
```

I claim that this is a confusing and/or wrong use of "Mandates:". My proposed resolution is simply to respecify the constructor as

```
explicit unique_ptr(type_identity_t<pointer> p) noexcept;
```

Constraints: `is_pointer_v<deleter_type>` is false and `is_default_constructible_v<deleter_type>` is true.

Preconditions: [...]

Effects: [...]

Postconditions: [...]

with no *Mandates:* or *Remarks:* elements at all.

[2022-01-29; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 20.11.1.3.1 [\[unique.ptr.single.general\]](#), class template `unique_ptr` synopsis, as indicated:

```
[...]
// 20.11.1.3.2 \[unique.ptr.single.ctor\], constructors
constexpr unique_ptr() noexcept;
explicit unique_ptr(type\_identity\_t<pointer> p) noexcept;
unique_ptr(type\_identity\_t<pointer> p, see below d1) noexcept;
unique_ptr(type\_identity\_t<pointer> p, see below d2) noexcept;
[...]
```

2. Modify 20.11.1.3.2 [\[unique.ptr.single.ctor\]](#) as indicated:

```
explicit unique_ptr(type\_identity\_t<pointer> p) noexcept;
```

-5- *Constraints:* `is_pointer_v<deleter_type>` is false and `is_default_constructible_v<deleter_type>` is true.

~~-6- *Mandates:* This constructor is not selected by class template argument deduction (12.2.2.9 [\[over.match.class.deduct\]](#)).~~

-7- *Preconditions:* [...]

-8- *Effects:* [...]

-9- *Postconditions:* [...]

```
unique_ptr(type\_identity\_t<pointer> p, const D& d) noexcept;
unique_ptr(type\_identity\_t<pointer> p, remove_reference_t<D>&& d) noexcept;
```

-10- *Constraints:* `is_constructible_v<D, decltype(d)>` is true.

~~-11- *Mandates:* These constructors are not selected by class template argument deduction (12.2.2.9 [\[over.match.class.deduct\]](#)).~~

-12- *Preconditions:* [...]

-13- *Effects:* [...]

-14- *Postconditions:* [...]

-15- *Remarks:* If `D` is a reference type, the second constructor is defined as deleted.

-16- *[Example 1: [...] — end example]*

[3643](#). Missing constexpr in `std::counted_iterator`

Section: 23.5.6.5 [\[counted.iter.nav\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2021-11-21 **Last modified:** 2022-01-30

Priority: Not Prioritized

Discussion:

One overload of `std::counted_operator::operator++` is not constexpr currently, which is seemly because of that a try-block (specified in 23.5.6.5 [\[counted.iter.nav\]](#)/4) is not allowed in a constexpr function until C++20. Given a try-block is allowed in a constexpr function in C++20, IMO this overload should also be constexpr.

MSVC STL has already added constexpr [at first](#). The situation of this overload is originally found by Casey Carter, but no LWG issue has been submitted.

[2022-01-30; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 23.5.6.1 [\[counted.iterator\]](#), class template `counted_iterator` synopsis, as indicated:

```
[...]
constexpr counted_iterator& operator++();
constexpr decltype(auto) operator++(int);
constexpr counted_iterator operator++(int)
    requires forward_iterator<I>;
constexpr counted_iterator& operator--()
    requires bidirectional_iterator<I>;
constexpr counted_iterator operator--(int)
    requires bidirectional_iterator<I>;
[...]
```

2. Modify 23.5.6.5 [\[counted.iter.nav\]](#) as indicated:

```
constexpr decltype(auto) operator++(int);

-3- Preconditions: length > 0.

-4- Effects: Equivalent to:

    --length;
    try { return current++; }
    catch(...) { ++length; throw; }
```

[3648](#). `format` should not print `bool` with `'c'`

Section: 20.20.2.2 [\[format.string.std\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Zhihao Yuan **Opened:** 2021-11-30 **Last modified:** 2022-01-30

Priority: Not Prioritized

View other [active issues](#) in `[format.string.std]`.

View all other [issues](#) in `[format.string.std]`.

Discussion:

[P1652R1](#) prints integral inputs as characters with `'c'` and preserves the wording to treat `bool` as a one-byte unsigned integer; this accidentally asks the implementation to cast `bool` into a 1-bit character if a user asks for the `'c'` presentation type.

Recent wording improvements made this implied behavior obvious.

[2021-12-04; Daniel comments]

This issue relates to LWG [3644](#).

[2022-01-30; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 20.20.2.2 [\[format.string.std\]](#), Table 67 `[tab:format.type.bool]`, as indicated:

Table 67 — Meaning of <i>type</i> options for <code>bool</code> <code>[tab:format.type.bool]</code>	
Type	Meaning
none, s	Copies textual representation, either <code>true</code> or <code>false</code> , to the output.
b, B, c ; d, o, x, X	As specified in Table 65 <code>[tab:format.type.int]</code> for the value <code>static_cast<unsigned char>(value)</code> .

[3649](#). `[fund.ts.v3]` Reinstate and bump `__cpp_lib_experimental_memory_resource` feature test macro

Section: 1.5 `[fund.ts.v3::general.feature.test]` **Status:** [Tentatively Ready](#) **Submitter:** Thomas Köppe **Opened:** 2021-12-03 **Last modified:** 2022-01-30

Priority: Not Prioritized

Discussion:

Addresses: `fund.ts.v3`

The rebase on C++17 in [P0996](#) had the effect of deleting the feature test macro `__cpp_lib_experimental_memory_resource`: the macro was ostensibly tied to the `memory_resource` facility that had become part of C++17, but we overlooked that there was a residual piece that has not been adopted in the IS, namely the `resource_adaptor` class template.

It is still useful to be able to detect the presence of `resource_adaptor`, so we should reinstate the feature test macro and bump its value.

[2022-01-30; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4853](#).

- 1. Modify 1.5 [fund.ts.v3::general.feature.test], Table 2, as indicated:

Table 2 — Significant features in this technical specification					
Doc. No.	Title	Primary Section	Macro Name Suffix	Value	Header
...					
N3916	Type-erased allocator for <code>std::function</code>	4.2	<code>function_erased_allocator</code>	201406	<experimental/functional>
N3916	Polymorphic Memory Resources	5.4 [fund.ts.v3::memory_resource.syn]	<code>memory_resources</code>	[new value]	<experimental/memory_resource>
N4282	The World's Dumbest Smart Pointer	8.12	<code>observer_ptr</code>	201411	<experimental/memory>
...					

3650. Are `std::basic_string`'s `iterator` and `const_iterator` constexpr iterators?

Section: 21.3.3.1 [basic.string.general] **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2021-12-04 **Last modified:** 2022-01-30

Priority: Not Prioritized

Discussion:

`std::vector`'s `iterator` and `const_iterator` are required to meet constexpr iterator requirements in C++20 per [P1004R2](#), but it seems that the similar wording is missing for `std::basic_string` in both [P0980R1](#) and the current working draft.

I think we should add a bullet "The types `iterator` and `const_iterator` meet the constexpr iterator requirements (23.3.1 [iterator.requirements.general])." to 21.3.3.1 [basic.string.general].

[2022-01-30; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4901](#).

- 1. Modify 21.3.3.1 [basic.string.general], as indicated:

- 3- In all cases, `[data(), data() + size())` is a valid range, `data() + size()` points at an object with value `charT()` (a "null terminator"), and `size() <= capacity()` is true.
- 4- A `size_type` parameter type in a `basic_string` deduction guide refers to the `size_type` member type of the type deduced by the deduction guide.

~~[-? The types `iterator` and `const_iterator` meet the constexpr iterator requirements (23.3.1 [iterator.requirements.general]).~~

3654. `basic_format_context::arg(size_t)` should be noexcept

Section: 20.20.6.4 [format.context] **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2021-12-26 **Last modified:** 2022-01-30

Priority: Not Prioritized

View all other [issues](#) in [format.context].

Discussion:

`basic_format_context::arg(size_t)` simply returns `args_.get(id)` to get the elements of `args_`, where the type of `args_` is `basic_format_args<basic_format_context>`. Since `basic_format_args`'s `get(size_t)` is `noexcept`, this function can also be `noexcept`.

[2022-01-30; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 20.20.6.4 [\[format.context\]](#) as indicated:

```
namespace std {
    template<class Out, class charT>
    class basic_format_context {
        basic_format_args<basic_format_context> args_; // exposition only
        Out out_; // exposition only
    public:
        using iterator = Out;
        using char_type = charT;
        template<class T> using formatter_type = formatter<T, charT>;

        basic_format_arg<basic_format_context> arg(size_t id) const noexcept;
        std::locale locale();

        iterator out();
        void advance_to(iterator it);
    };
}
```

[...]

```
basic_format_arg<basic_format_context> arg(size_t id) const noexcept;
```

-5- Returns: `args_.get(id)`.

3657. `std::hash<std::filesystem::path>` is not enabled

Section: 29.12.6 [\[fs.class.path\]](#) Status: [Tentatively Ready](#) Submitter: Jiang An Opened: 2022-01-02 Last modified: 2022-01-30

Priority: Not Prioritized

View all other [issues](#) in [\[fs.class.path\]](#).

Discussion:

The `hash` support of `std::filesystem::path` is provided by `std::filesystem::hash_value`, but the specialization `std::hash<std::filesystem::path>` is currently disabled. IMO the specialization should be enabled, and its `operator()` should return the same value as `hash_value`.

[2022-01-15; Daniel provides wording]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4901](#).

1. Modify 29.12.4 [\[fs.filesystem.syn\]](#), header `<filesystem>` synopsis, as indicated:

```
#include <compare> // see 17.11.1 \[compare.syn\]

namespace std::filesystem {
    // 29.12.6 \[fs.class.path\], paths
    class path;

    // 29.12.6.8 \[fs.path.nonmember\], path non-member functions
    void swap(path& lhs, path& rhs) noexcept;
    size_t hash_value(const path& p) noexcept;

    [...]
}

// \[fs.path.hash\], hash support
namespace std {
    template<class T> struct hash;
    template<> struct hash<filesystem::path>;
```


};

2. Following subclause 29.12.6.8 [fs.path.nonmember], introduce a new subclause [fs.path.hash], as indicated:

29.12.6.? Hash support [fs.path.hash]

```
template<> struct hash<filesystem::path>;
```

[-? For an object p of type filesystem::path, hash<filesystem::path>.(p) shall evaluate to the same result as hash_value(p).]

[2022-01-18; Daniel improves wording based on reflector discussion feedback]

[2022-01-30; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 29.12.4 [fs.filesystem.syn], header <filesystem> synopsis, as indicated:

```
#include <compare> // see 17.11.1 [compare.syn]

namespace std::filesystem {
    // 29.12.6 [fs.class.path], paths
    class path;

    // 29.12.6.8 [fs.path.nonmember], path non-member functions
    void swap(path& lhs, path& rhs) noexcept;
    size_t hash_value(const path& p) noexcept;

    [...]
}

// [fs.path.hash], hash support
namespace std {
    template<class T> struct hash;
    template<> struct hash<filesystem::path>;
};
```

2. Following subclause 29.12.6.8 [fs.path.nonmember], introduce a new subclause [fs.path.hash], as indicated:

29.12.6.? Hash support [fs.path.hash]

```
template<> struct hash<filesystem::path>;
```

[-? For an object p of type filesystem::path, hash<filesystem::path>.(p) evaluates to the same result as filesystem::hash_value(p).]

3660. iterator_traits<common_iterator>::pointer should conform to §[iterator.traits]

Section: 23.5.4.2 [common.iter.types] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2022-01-20 **Last modified:** 2022-01-30

Priority: Not Prioritized

Discussion:

23.3.2.3 [iterator.traits]/1 says:

[...] In addition, the types

```
iterator_traits<I>::pointer
iterator_traits<I>::reference
```

shall be defined as the iterator's pointer and reference types; that is, for an iterator object a of class type, the same type as `decltype(a.operator->())` and `decltype(*a)`, respectively. The type `iterator_traits<I>::pointer` shall be void for an iterator of class type I that does not support `operator->`. [...]

23.5.4.2 [common.iter.types]/1 slightly contradicts this:

The nested *typedef-names* of the specialization of `iterator_traits` for `common_iterator<I, S>` are defined as follows.

[...]

(1.3) — If the expression `a.operator->()` is well-formed, where `a` is an lvalue of type `const common_iterator<I, S>`, then pointer denotes the type of that expression. Otherwise, pointer denotes void.

"The type of `a.operator->()`" is not necessarily the same as `decltype(a.operator->())`: when the expression is an lvalue or xvalue of type `T`, "the type of `a.operator->()`" is `T` but `decltype(a.operator->())` is either `T&` or `T&&`. An implementation therefore cannot conform to the requirements of both cited paragraphs for some specializations of `common_iterator`.

The most likely explanation for this contradiction is that the writer of the phrase "type of `a.operator->()`" was not cognizant of the difference in meaning and intended to actually write `decltype(a.operator->())`.

[2022-01-30; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4901](#).

[Drafting Note: The wording change below includes an additional drive-by fix that ensures that the last sentence "Otherwise, pointer denotes void" cannot be misinterpreted (due to the leading "If") to apply also for a situation when the outcome of the expression `a.operator->()` for a non-const `common_iterator<I, S>` is reflected upon.]

1. Modify 23.5.4.2 [\[common.iter.types\]](#) as indicated:

-1- The nested *typedef-names* of the specialization of `iterator_traits` for `common_iterator<I, S>` are defined as follows.

[...]

(1.3) — Let `a` denote an lvalue of type `const common_iterator<I, S>`. If the expression `a.operator->()` is well-formed, where `a` is an lvalue of type `const common_iterator<I, S>`, then pointer denotes `decltype(a.operator->())` the type of that expression. Otherwise, pointer denotes void.

[3661](#). `constexpr atomic<shared_ptr<T>> a(nullptr);` should work

Section: 31.8.7.2 [\[util.smartptr.atomic.shared\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2022-01-21 **Last modified:** 2022-01-30

Priority: Not Prioritized

Discussion:

All the following are valid except for the last line:

```
constexpr int i1{};
constexpr std::atomic<int> a1{};
constexpr int i2{0};
constexpr std::atomic<int> a2{0};
constexpr std::shared_ptr<int> i3{};
constexpr std::atomic<std::shared_ptr<int>> a3{};
constexpr std::shared_ptr<int> i4{nullptr};
constexpr std::atomic<std::shared_ptr<int>> a4{nullptr}; // error
```

The initializer for `a4` will create a `shared_ptr<int>` temporary (using the same constructor as `i4`) but then try to use `atomic(shared_ptr<int>)` which is not `constexpr`.

This is an unnecessary inconsistency in the API for `atomic<shared_ptr<T>>` that can easily be fixed. The proposed resolution has been implemented in `libstdc++`.

There is no need to also change `atomic<weak_ptr<T>>` because `weak_ptr` doesn't have a constructor taking `nullptr`.

[2022-01-30; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 31.8.7.2 [\[util.smartptr.atomic.shared\]](#), class template partial specialization `atomic<shared_ptr<T>>` synopsis, as indicated:

```
[...]
constexpr atomic() noexcept;
constexpr atomic(nullptr t) noexcept : atomic() {}
atomic(shared_ptr<T> desired) noexcept;
```

```
atomic(const atomic&) = delete;  
void operator=(const atomic&) = delete;  
[...]
```